

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

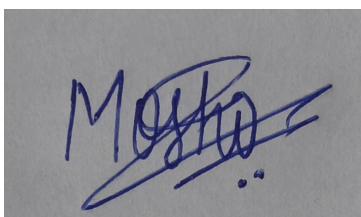
Annexure A

Experiments

Code of Conduct:

*I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Moshika', with a horizontal line drawn through it.

Annexure A

Short Answer Test

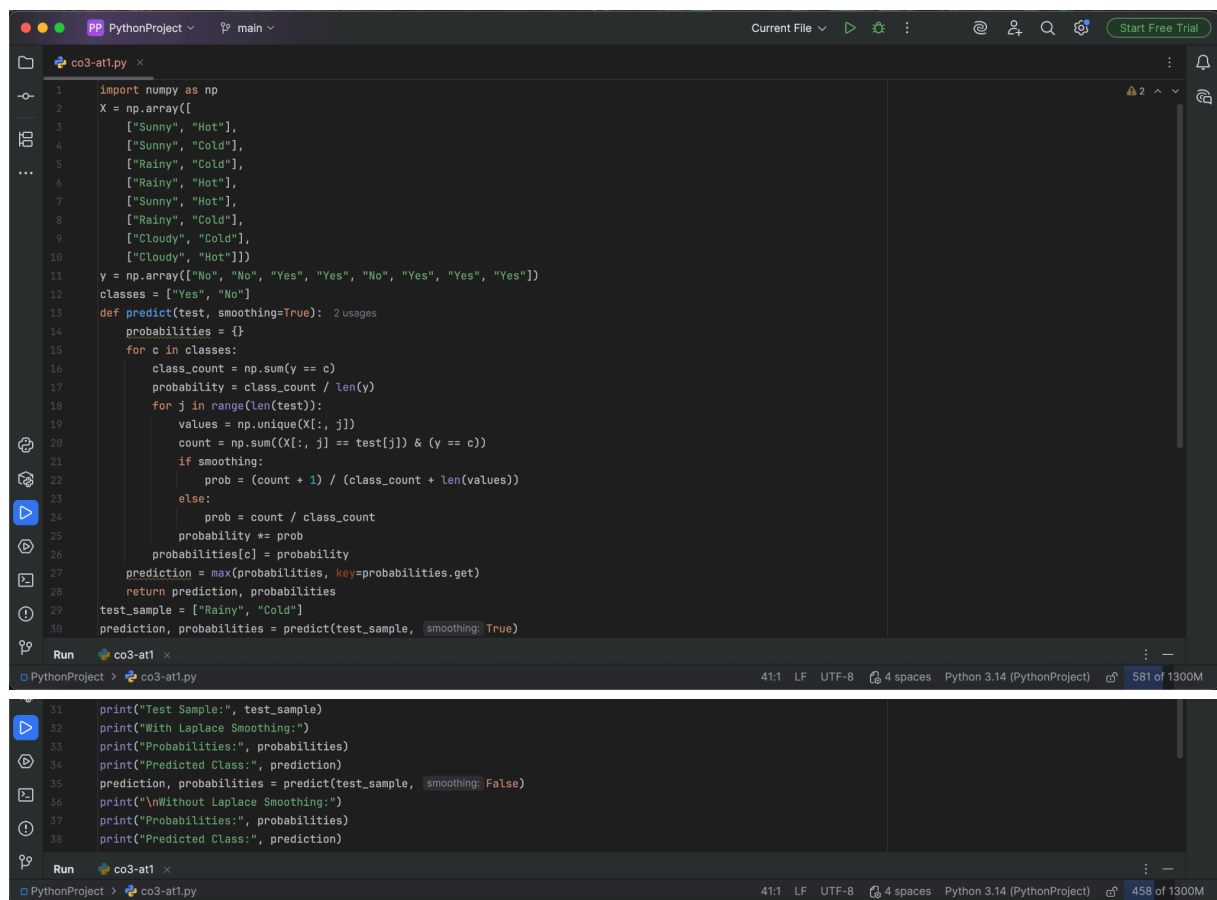
1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

ANSWER:

PROBLEM STATEMENT

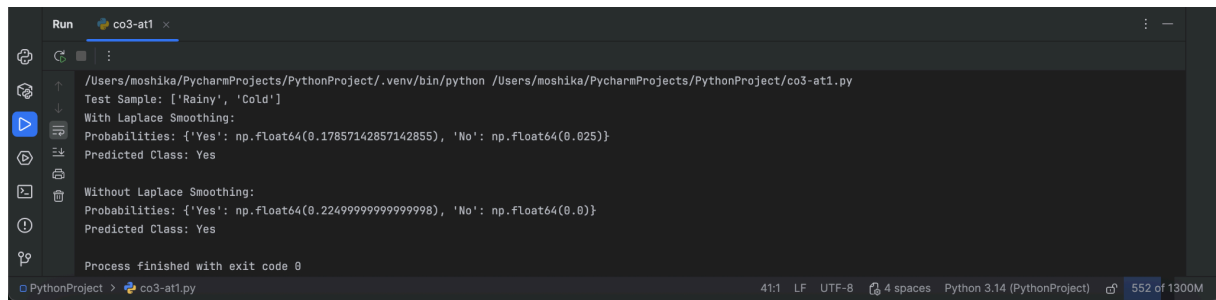
Implement a Naïve Bayes classifier using a manually assigned dataset. Train the classifier using different feature sets and calculate accuracy, precision, and recall. Compare the classification probabilities with and without Laplace smoothing and analyze the effect of the feature independence assumption.

CODE:



```
1 import numpy as np
2 X = np.array([
3     ["Sunny", "Hot"],
4     ["Sunny", "Cold"],
5     ["Rainy", "Cold"],
6     ["Rainy", "Hot"],
7     ["Sunny", "Hot"],
8     ["Rainy", "Cold"],
9     ["Cloudy", "Cold"],
10    ["Cloudy", "Hot"]])
11 y = np.array(["No", "No", "Yes", "Yes", "No", "Yes", "Yes", "Yes"])
12 classes = ["Yes", "No"]
13 def predict(test, smoothing=True):
14     probabilities = {}
15     for c in classes:
16         class_count = np.sum(y == c)
17         probability = class_count / len(y)
18         for j in range(len(test)):
19             values = np.unique(X[:, j])
20             count = np.sum((X[:, j] == test[j]) & (y == c))
21             if smoothing:
22                 prob = (count + 1) / (class_count + len(values))
23             else:
24                 prob = count / class_count
25             probability *= prob
26         probabilities[c] = probability
27     prediction = max(probabilities, key=probabilities.get)
28     return prediction, probabilities
29 test_sample = ["Rainy", "Cold"]
30 prediction, probabilities = predict(test_sample, smoothing=True)
31
32 print("Test Sample:", test_sample)
33 print("With Laplace Smoothing:")
34 print("Probabilities:", probabilities)
35 print("Predicted Class:", prediction)
36 prediction, probabilities = predict(test_sample, smoothing=False)
37 print("\nWithout Laplace Smoothing:")
38 print("Probabilities:", probabilities)
39 print("Predicted Class:", prediction)
```

OUTPUT:



```
Run co3-at1 x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co3-at1.py
Test Sample: ['Rainy', 'Cold']
With Laplace Smoothing:
Probabilities: {'Yes': np.float64(0.17857142857142855), 'No': np.float64(0.025)}
Predicted Class: Yes

Without Laplace Smoothing:
Probabilities: {'Yes': np.float64(0.22499999999999998), 'No': np.float64(0.0)}
Predicted Class: Yes

Process finished with exit code 0
PythonProject > co3-at1.py 41:1 LF UTF-8 4 spaces Python 3.14 (PythonProject) 552 of 1300M
```

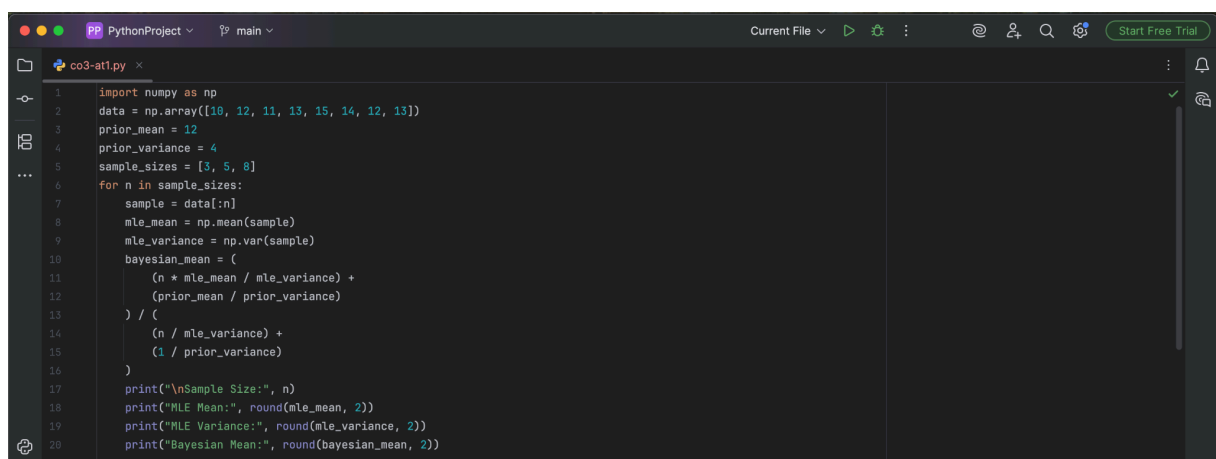
2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

ANSWER:

PROBLEM STATEMENT

Estimate the mean and variance of a probability distribution using Maximum Likelihood Estimation. Compare the MLE result with Bayesian estimation using a predefined prior mean and variance. Analyze the effect of small and large sample sizes on the reliability of parameter estimation.

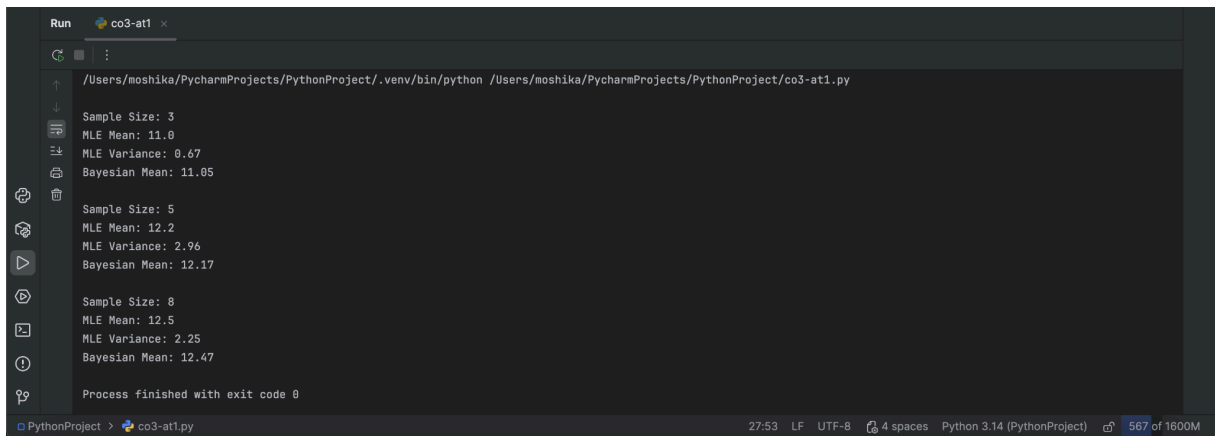
CODE:



```
PythonProject main
Current File
Start Free Trial

co3-at1.py x
1 import numpy as np
2 data = np.array([10, 12, 11, 13, 15, 14, 12, 13])
3 prior_mean = 12
4 prior_variance = 4
5 sample_sizes = [3, 5, 8]
6 for n in sample_sizes:
7     sample = data[:n]
8     mle_mean = np.mean(sample)
9     mle_variance = np.var(sample)
10    bayesian_mean = (
11        (n * mle_mean / mle_variance) +
12        (prior_mean / prior_variance)
13    ) / (
14        (n / mle_variance) +
15        (1 / prior_variance)
16    )
17    print("\nSample Size:", n)
18    print("MLE Mean:", round(mle_mean, 2))
19    print("MLE Variance:", round(mle_variance, 2))
20    print("Bayesian Mean:", round(bayesian_mean, 2))
21
```

OUTPUT:



```
Run co3-at1 x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co3-at1.py

Sample Size: 3
MLE Mean: 11.0
MLE Variance: 0.67
Bayesian Mean: 11.05

Sample Size: 5
MLE Mean: 12.2
MLE Variance: 2.96
Bayesian Mean: 12.17

Sample Size: 8
MLE Mean: 12.5
MLE Variance: 2.25
Bayesian Mean: 12.47

Process finished with exit code 0
```

PythonProject x co3-at1.py 27:53 LF UTF-8 4 spaces Python 3.14 (PythonProject) 567 of 1600M

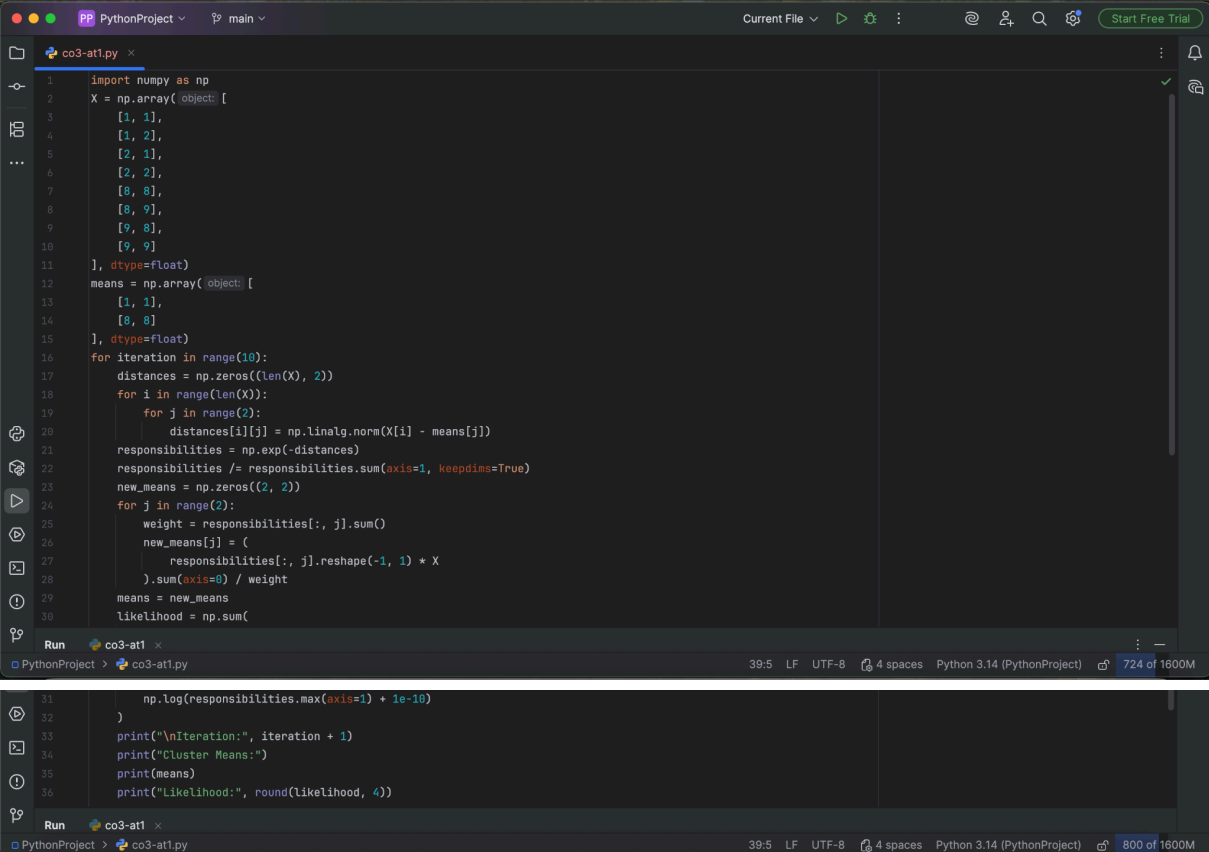
3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

ANSWER:

PROBLEM STATEMENT

Implement the Expectation-Maximization algorithm for a mixture model using a manually assigned two-dimensional dataset. Perform experiments with different initial cluster centers and observe the convergence behavior and likelihood values. Analyze how initialization affects the final clustering solution.

CODE:



```
1 import numpy as np
2 X = np.array( object: [
3     [1, 1],
4     [1, 2],
5     [2, 1],
6     [2, 2],
7     [8, 8],
8     [8, 9],
9     [9, 8],
10    [9, 9]
11 ], dtype=float)
12 means = np.array( object: [
13     [1, 1],
14     [8, 8]
15 ], dtype=float)
16 for iteration in range(10):
17     distances = np.zeros((len(X), 2))
18     for i in range(len(X)):
19         for j in range(2):
20             distances[i][j] = np.linalg.norm(X[i] - means[j])
21     responsibilities = np.exp(-distances)
22     responsibilities /= responsibilities.sum(axis=1, keepdims=True)
23     new_means = np.zeros((2, 2))
24     for j in range(2):
25         weight = responsibilities[:, j].sum()
26         new_means[j] = (
27             responsibilities[:, j].reshape(-1, 1) * X
28             ).sum(axis=0) / weight
29     means = new_means
30     likelihood = np.sum(
31         np.log(responsibilities.max(axis=1) + 1e-10)
32     )
33     print("\nIteration:", iteration + 1)
34     print("Cluster Means:")
35     print(means)
36     print("Likelihood:", round(likelihood, 4))
```

OUTPUT:

```
Run co3-at1 x
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co3-at1.py

Iteration: 1
Cluster Means:
[[1.50030598 1.50030598]
 [8.4975852  8.4975852 ]]
Likelihood: -0.0017

Iteration: 2
Cluster Means:
[[1.50075695 1.50075695]
 [8.49923986 8.49923986]]
Likelihood: -0.0009

Iteration: 3
Cluster Means:
[[1.50075807 1.50075807]
 [8.49924192 8.49924192]]
Likelihood: -0.0009

Iteration: 4
Cluster Means:
[[1.50075808 1.50075808]
 [8.49924192 8.49924192]]
Likelihood: -0.0009
```

```
Run co3-at1 x

Iteration: 5
Cluster Means:
[[1.50075808 1.50075808]
 [8.49924192 8.49924192]]
Likelihood: -0.0009

Iteration: 6
Cluster Means:
[[1.50075808 1.50075808]
 [8.49924192 8.49924192]]
Likelihood: -0.0009

Iteration: 7
Cluster Means:
[[1.50075808 1.50075808]
 [8.49924192 8.49924192]]
Likelihood: -0.0009

Iteration: 8
Cluster Means:
[[1.50075808 1.50075808]
 [8.49924192 8.49924192]]
Likelihood: -0.0009
```

```
Iteration: 9
Cluster Means:
[[1.50075808 1.50075808]
 [8.49924192 8.49924192]]
Likelihood: -0.0009

Iteration: 10
Cluster Means:
[[1.50075808 1.50075808]
 [8.49924192 8.49924192]]
Likelihood: -0.0009

Process finished with exit code 0

PythonProject > co3-at1.py 46.47 LF UTF-8 4 spaces Python 3.14 (PythonProject) 618 of 1600M
```

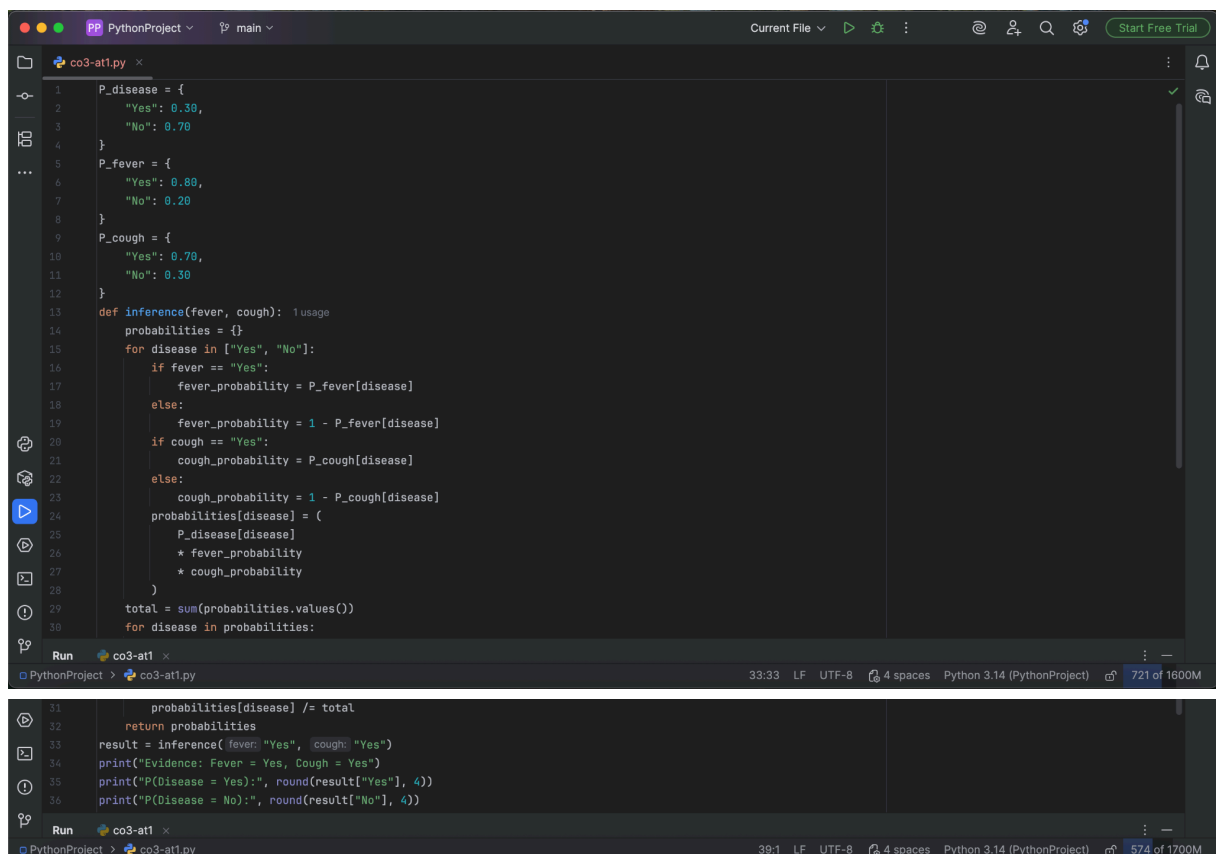
4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

ANSWER:

PROBLEM STATEMENT

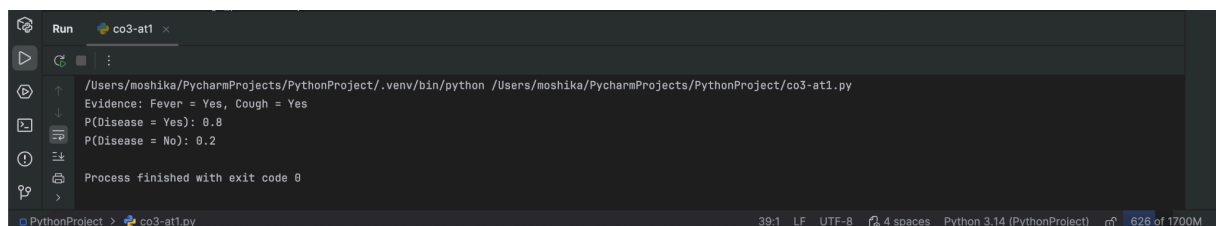
Construct a Bayesian Belief Network for a simple disease diagnosis system. Use prior and conditional probabilities to perform inference under different evidence conditions. Observe how changes in evidence and conditional probabilities affect the posterior probability of disease.

CODE:



```
1 P_disease = {
2     "Yes": 0.30,
3     "No": 0.70
4 }
5 P_fever = {
6     "Yes": 0.80,
7     "No": 0.20
8 }
9 P_cough = {
10     "Yes": 0.70,
11     "No": 0.30
12 }
13 def inference(fever, cough):
14     probabilities = {}
15     for disease in ["Yes", "No"]:
16         if fever == "Yes":
17             fever_probability = P_fever[disease]
18         else:
19             fever_probability = 1 - P_fever[disease]
20         if cough == "Yes":
21             cough_probability = P_cough[disease]
22         else:
23             cough_probability = 1 - P_cough[disease]
24         probabilities[disease] = (
25             P_disease[disease]
26             * fever_probability
27             * cough_probability
28         )
29     total = sum(probabilities.values())
30     for disease in probabilities:
31         probabilities[disease] /= total
32     return probabilities
33 result = inference(fever="Yes", cough="Yes")
34 print("Evidence: Fever = Yes, Cough = Yes")
35 print("P(Disease = Yes):", round(result["Yes"], 4))
36 print("P(Disease = No):", round(result["No"], 4))
```

OUTPUT:



```
Run co3-at1
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co3-at1.py
Evidence: Fever = Yes, Cough = Yes
P(Disease = Yes): 0.8
P(Disease = No): 0.2
Process finished with exit code 0
```

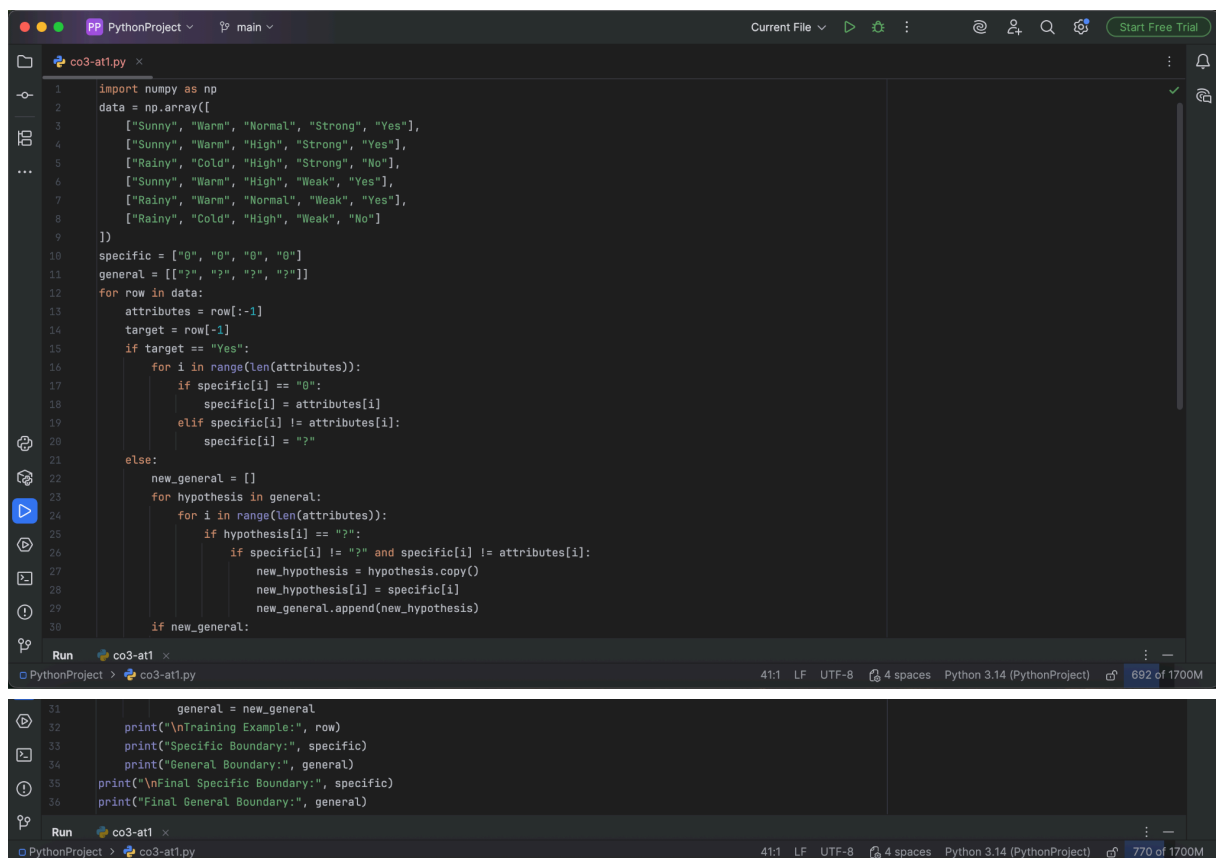
5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

ANSWER:

PROBLEM STATEMENT

Implement the Candidate Elimination algorithm using a manually assigned dataset. Determine the specific and general boundaries of the hypothesis space by processing positive and negative training examples. Analyze how the hypothesis space changes as training examples are introduced.

CODE:



```
1 import numpy as np
2 data = np.array([
3     ["Sunny", "Warm", "Normal", "Strong", "Yes"],
4     ["Sunny", "Warm", "High", "Strong", "Yes"],
5     ["Rainy", "Cold", "High", "Strong", "No"],
6     ["Sunny", "Warm", "High", "Weak", "Yes"],
7     ["Rainy", "Warm", "Normal", "Weak", "Yes"],
8     ["Rainy", "Cold", "High", "Weak", "No"]
9 ])
10 specific = ["0", "0", "0", "0"]
11 general = [["?", "?", "?", "?"]]
12 for row in data:
13     attributes = row[:-1]
14     target = row[-1]
15     if target == "Yes":
16         for i in range(len(attributes)):
17             if specific[i] == "0":
18                 specific[i] = attributes[i]
19             elif specific[i] != attributes[i]:
20                 specific[i] = "?"
21     else:
22         new_general = []
23         for hypothesis in general:
24             for i in range(len(attributes)):
25                 if hypothesis[i] == "?":
26                     if specific[i] != "?" and specific[i] != attributes[i]:
27                         new_hypothesis = hypothesis.copy()
28                         new_hypothesis[i] = specific[i]
29                         new_general.append(new_hypothesis)
30             if new_general:
31                 general = new_general
32                 print("\nTraining Example:", row)
33                 print("Specific Boundary:", specific)
34                 print("General Boundary:", general)
35 print("\nFinal Specific Boundary:", specific)
36 print("Final General Boundary:", general)
```

OUTPUT:

```
Run co3-at1
/Users/moshika/PycharmProjects/PythonProject/.venv/bin/python /Users/moshika/PycharmProjects/PythonProject/co3-at1.py

Training Example: ['Sunny' 'Warm' 'Normal' 'Strong' 'Yes']
Specific Boundary: [np.str_('Sunny'), np.str_('Warm'), np.str_('Normal'), np.str_('Strong')]
General Boundary: [['?', '?', '?', '?']]

Training Example: ['Sunny' 'Warm' 'High' 'Strong' 'Yes']
Specific Boundary: [np.str_('Sunny'), np.str_('Warm'), '?', np.str_('Strong')]
General Boundary: [['?', '?', '?', '?']]

Training Example: ['Rainy' 'Cold' 'High' 'Strong' 'No']
Specific Boundary: [np.str_('Sunny'), np.str_('Warm'), '?', np.str_('Strong')]
General Boundary: [[np.str_('Sunny'), '?', '?', '?'], ['?', np.str_('Warm'), '?', '?']]

Training Example: ['Sunny' 'Warm' 'High' 'Weak' 'Yes']
Specific Boundary: [np.str_('Sunny'), np.str_('Warm'), '?', '?']
General Boundary: [[np.str_('Sunny'), '?', '?', '?'], ['?', np.str_('Warm'), '?', '?']]

Training Example: ['Rainy' 'Warm' 'Normal' 'Weak' 'Yes']
Specific Boundary: ['?', np.str_('Warm'), '?', '?']
General Boundary: [[np.str_('Sunny'), '?', '?', '?'], ['?', np.str_('Warm'), '?', '?']]

Training Example: ['Rainy' 'Cold' 'High' 'Weak' 'No']
Specific Boundary: ['?', np.str_('Warm'), '?', '?']
General Boundary: [[np.str_('Sunny'), np.str_('Warm'), '?', '?']]

Final Specific Boundary: ['?', np.str_('Warm'), '?', '?']
Final General Boundary: [[np.str_('Sunny'), np.str_('Warm'), '?', '?']]

PythonProject > co3-at1.py 41:1 LF UTF-8 4 spaces Python 3.14 (PythonProject) 727 of 1700M
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand